

基于 YOLO11 识别红绿灯

三. 模型部署、验证及应用

4、模型部署及验证

远程连接到小车，在小车 userdata 目录下，新建 yolo11 文件夹。

把 yolo11n_detect_bayese_640x640_nv12_modified.bin、

predefined_classes.txt 上传到该目录下，

运行（这是一条命令）

```
cp /opt/tros/humble/lib/dnn_node_example/config/yolov10workconfig.json
/userdata/yolo11/yolov11workconfig.json
```

打开 yolov11workconfig.json，修改

```
* yolov11workconf...
1 {
2   "model_file": "/userdata/yolo11/yolo11n_detect_bayese_640x640_nv12_modified.bin",
3   "task_num": 4,
4   "dnn_parser": "yolov11",
5   "model_output_count": 6,
6   "reg_max": 16,
7   "class_num": 3,
8   "cls_names_list": "/userdata/yolo11/predefined_classes.txt",
9   "strides": [8, 16, 32],
10  "score_threshold": 0.25,
11  "top_k": 300,
12  "output_order": [0,1,2,3,4,5]
13 }
14 }
```

上传几张红绿灯图片到 yolo11 目录下，运行

export CAM_TYPE=fb

```
ros2 launch dnn_node_example dnn_node_example.launch.py
```

```
dnn_example_config_file:=/userdata/yolo11/yolov11workconfig.json
```

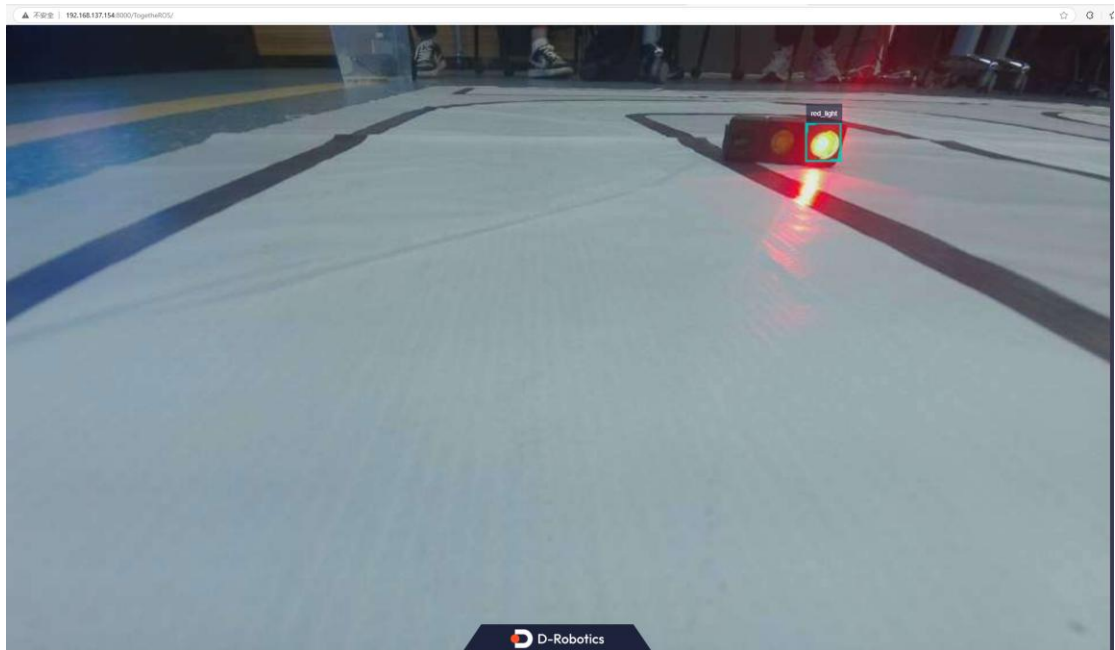
```
publish_image_source:=/userdata/yolo11/照片名
```

***黄色的是一条指令**

```
root@ubuntu:~# export CAM_TYPE=fb
root@ubuntu:~# ros2 launch dnn_node_example dnn_node_example.launch.py dnn_example_config_file:=/userdata/yolo11/yolov11workconfig.json pub
/target.jpg

[INFO] [launch]: All log files can be found below /root/.ros/log/2025-04-13-14-25-21-709200-ubuntu-4269
[INFO] [launch]: Default logging verbosity is set to INFO
dnn_node_example_path is /opt/tros/humble/lib/dnn_node_example
cp cmd is cp -r /opt/tros/humble/lib/dnn_node_example/config .
camera type is fb
using feedback
Robot shm pkg enables zero-copy with fastrtps profiles file: /opt/tros/humble/lib/hobot_shm/config/shm_fastdds.xml
Hobot shm pkg sets RMW_FASTRTS_USE_QOS_FROM_XML is 1
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
launch webserver
pwd path is /root
webserver_path is /opt/tros/humble/lib/websocket/websocket
launch webserver cmd is ./sbin/nginx -p
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
env of RMW_FASTRTS_USE_QOS_FROM_XML is 1, ignore env setting
```

在浏览器，输入小车 IP:8000 观察



中断程序后，输入

`export CAM_TYPE=usb`

`ros2 launch dnn_node_example dnn_node_example.launch.py`

`dnn_example_config_file:=/userdata/yolo11/yolov11workconfig.json`

***黄色的是一条指令**

```
root@ubuntu:~# export CAM_TYPE=mipi
root@ubuntu:~# ros2 launch dnn_node_example dnn_node_example.launch.py dnn_example_config_file:=/userdata/yolo11/yolov11workconfig.json
[INFO] [launch]: All log files can be found below /root/.ros/log/2025-04-13-14-31-26-855377-ubuntu-11149
[INFO] [launch]: Default logging verbosity is set to INFO
dnn_node_example_path is /opt/tros/humble/lib/dnn_node_example
cp_cmd is cp -r /opt/tros/humble/lib/dnn_node_example/config .
camera type is mipi
using mipi cam
Hobot shm pkg enables zero-copy with fasttrtps profiles file: /opt/tros/humble/lib/hobot_shm/config/shm_fastdds.xml
Hobot shm pkg sets RMW FASTRTPS USE_QOS_FROM_XML: 1
config file path is /opt/tros/humble/lib/mipi_cam/config/
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
webserver has launch
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
config file path is /opt/tros/humble/lib/mipi_cam/config/
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
env of RMW FASTRTPS USE_QOS_FROM_XML is 1, ignore env setting
webserver has launch
```

把红绿灯放在小车前面，从网页观察



开另一个窗口，输入

`ros2 topic echo /hobot_dnn_detection`

```

time_ms_duration: 12.0
targets:
- type: red_light
  track_id: 0
  rois:
  - type: red_light
    rect:
      x_offset: 37
      y_offset: 34
      height: 91
      width: 84
      do_rectify: false
      confidence: 0.5831394791603088
    attributes: []
    points: []
    captures: []
- type: red_light
  track_id: 0
  rois:
  - type: red_light
    rect:
      x_offset: 70
      y_offset: 45
      height: 66
      width: 49
      do_rectify: false
      confidence: 0.2695736289024353
    attributes: []
    points: []
    captures: []
disappeared_targets: []

```

5、实际使用

- 1) 在 /userdata/dev_ws/install/lanedetection/share/lanedetection/目录下，新建 launch 文件夹，上传 dnn_node_yolo.launch.py 到这个目录下

- 2) 第 1 个窗口，启动底盘、摄像头和激光雷达

```
ros2 launch originbot bringup originbot.launch.py use_camera:=true use_lidar:=true
```

- 3) 第 2 个窗口，启动 yolo 识别红绿灯的程序

```
ros2 launch lanedetection dnn_node_yolo.launch.py
```

- 4) 第 3 个窗口，查看话题（topic）和数据

```
ros2 topic list
```

```
root@ubuntu:~# ros2 topic list
/camera_info
/cmd_vel
/hbmem_img0x2107020403080120021609080306270d
/odom
/originbot_status
/parameter_events
/rosout
/scan
/tf
/tf_static
/traffic_sign
/ydlidar_ros2_driver_node/transition_event
root@ubuntu:~#
```

查看数据

```
ros2 topic echo /traffic_sign
```

```
targets:
- type: "red_light\r"
  track_id: 0
  rois:
  - type: "red_light\r"
    rect:
      x_offset: 612
      y_offset: 55
      height: 41
      width: 41
      do_rectify: false
    confidence: 0.41854217648506165
  attributes: []
  points: []
  captures: []
disappeared_targets: []
```

红灯 red_light

```
targets:
- type: green_light
  track_id: 0
  rois:
  - type: green_light
    rect:
      x_offset: 652
      y_offset: 67
      height: 32
      width: 31
      do_rectify: false
    confidence: 0.7247857451438904
  attributes: []
  points: []
  captures: []
disappeared_targets: []
```

绿灯 green_light

查看数据类型

`ros2 topic type /traffic_sign`

```
^Croot@ubuntu:~# ros2 topic type /traffic_sign
ai_msgs/msg/PerceptionTargets
root@ubuntu:~#
```

```
root@ubuntu:/userdata# python view_traffic.py
[INFO] [1744528531.407160094] [View_Traffic]: Traffic_sign is OK
green_light
green_light
green_light
green_light
green_light
```

附录 view_traffic.py

```
1. #!/usr/bin/env python
2. # -*- coding: UTF-8 -*-
3.
4. import rclpy
5. from rclpy.node import Node
6. from std_msgs.msg import Bool
7. from ai_msgs.msg import PerceptionTargets
8.
9. class ViewTraffic(Node):
10.     def __init__(self, name):
11.         super().__init__(name)
12.         self.get_logger().info("Traffic_sign is OK")
13.
14.         self.traffic_sub= self.create_subscription(PerceptionTargets,
15.             "/traffic_sign", self.traffic_callback, 10)
16.
17.     def traffic_callback(self, msg):
```

```

18.         for boxes in msg.targets:
19.             if boxes.rois[0].confidence > 0.3 and boxes.rois[0].type == 'green_light\r':
20.                 print('green_light')
21.             elif boxes.rois[0].confidence > 0.3 and boxes.rois[0].type == 'red_light':
22.                 print('red_light')
23.
24. def main(args=None):
25.     rclpy.init(args=args)
26.     trafficview = ViewTraffic("View_Traffic")
27.     rclpy.spin(trafficview)
28.     trafficview.destroy_node()
29.     rclpy.shutdown()
30.
31. if __name__ == '__main__':
32.     main()

```

附录 dnn_node_yolo.launch.py

```

1. import os
2. from launch import LaunchDescription
3. from launch_ros.actions import Node
4.
5. def generate_launch_description():
6.     dnn_node = Node(
7.         package='dnn_node_example',
8.         executable='example',
9.         name='dnn_node',
10.        output='screen',
11.        arguments=[
12.            '--ros-args', '--log-level', 'warn',
13.            '--ros-args', '-p', 'feed_type:=1',
14.            '--ros-args', '-p', 'dump_render_img:=0',
15.            '--ros-args', '-p', 'is_shared_mem_sub:=1',
16.            '--ros-args', '-p', 'config_file:=/userdata/yolo11/yolov11workconfig.json',
17.            '--ros-args', '-p', 'msg_pub_topic_name:=traffice_sign'
18.        ]
19.    )
20.    return LaunchDescription([
21.        dnn_node,
22.    ])

```

附录 pic_seg.py

```

1. import os
2. import shutil
3. import random
4. # random.seed(0) #随机种子, 可自选开启
5. def split_data(file_path, label_path, new_file_path, train_rate, val_rate, test_rate):

```

```

6.     images = os.listdir(file_path)
7.     labels = os.listdir(label_path)
8.     images_no_ext = {os.path.splitext(image)[0]: image for image in images}
9.     labels_no_ext = {os.path.splitext(label)[0]: label for label in labels}
10.    matched_data = [(img, images_no_ext[img], labels_no_ext[img]) for img in
        images_no_ext if img in labels_no_ext]
11.    unmatched_images = [img for img in images_no_ext if img not in labels_no
        _ext]
12.    unmatched_labels = [label for label in labels_no_ext if label not in ima
        ges_no_ext]
13.    if unmatched_images:
14.        print("未匹配的图片文件:")
15.        for img in unmatched_images:
16.            print(images_no_ext[img])
17.    if unmatched_labels:
18.        print("未匹配的标签文件:")
19.        for label in unmatched_labels:
20.            print(labels_no_ext[label])
21.    random.shuffle(matched_data)
22.    total = len(matched_data)
23.    train_data = matched_data[:int(train_rate * total)]
24.    val_data = matched_data[int(train_rate * total):int((train_rate + val_ra
        te) * total)]
25.    test_data = matched_data[int((train_rate + val_rate) * total):]
26.    # 处理训练集
27.    for img_name, img_file, label_file in train_data:
28.        old_img_path = os.path.join(file_path, img_file)
29.        old_label_path = os.path.join(label_path, label_file)
30.        new_img_dir = os.path.join(new_file_path, 'train', 'images')
31.        new_label_dir = os.path.join(new_file_path, 'train', 'labels')
32.        os.makedirs(new_img_dir, exist_ok=True)
33.        os.makedirs(new_label_dir, exist_ok=True)
34.        shutil.copy(old_img_path, os.path.join(new_img_dir, img_file))
35.        shutil.copy(old_label_path, os.path.join(new_label_dir, label_file))
36.    # 处理验证集
37.    for img_name, img_file, label_file in val_data:
38.        old_img_path = os.path.join(file_path, img_file)
39.        old_label_path = os.path.join(label_path, label_file)
40.        new_img_dir = os.path.join(new_file_path, 'val', 'images')
41.        new_label_dir = os.path.join(new_file_path, 'val', 'labels')
42.        os.makedirs(new_img_dir, exist_ok=True)
43.        os.makedirs(new_label_dir, exist_ok=True)
44.        shutil.copy(old_img_path, os.path.join(new_img_dir, img_file))
45.        shutil.copy(old_label_path, os.path.join(new_label_dir, label_file))

```

```
46.     # 处理测试集
47.     for img_name, img_file, label_file in test_data:
48.         old_img_path = os.path.join(file_path, img_file)
49.         old_label_path = os.path.join(label_path, label_file)
50.         new_img_dir = os.path.join(new_file_path, 'test', 'images')
51.         new_label_dir = os.path.join(new_file_path, 'test', 'labels')
52.         os.makedirs(new_img_dir, exist_ok=True)
53.         os.makedirs(new_label_dir, exist_ok=True)
54.         shutil.copy(old_img_path, os.path.join(new_img_dir, img_file))
55.         shutil.copy(old_label_path, os.path.join(new_label_dir, label_file))
56.     print("数据集已划分完成")
57. if __name__ == '__main__':
58.     file_path = r"c:\Users\hchuang\pic_ws\VOC2007\JPEGImages" # 数据图片文件
                           夹绝对路径
59.     label_path = r'c:\Users\hchuang\pic_ws\VOC2007\Annotations'# 标签文件夹绝
                           对路径
60.     new_file_path = r"c:\Users\hchuang\ultralytics\traffices"# 新数据存放位置
61.     split_data(file_path, label_path, new_file_path, train_rate=0.8, val_rat
e=0.1, test_rate=0.1) #分割比例
```